

Assignment 2: Adversarial Search (Oxono)

Douaa El Mandili ; Edouard Lacheron
G090

1 Introduction

Dans le cadre de ce projet, nous avons développé un agent pour le jeu Oxono en utilisant des techniques de recherche adversariale. L'objectif est de construire un agent capable de jouer de manière compétitive face à différents types d'adversaires.

Nous avons choisi d'utiliser l'algorithme Alpha-Beta, qui permet d'explorer efficacement les coups possibles tout en réduisant le nombre de calculs nécessaires. Cependant, les performances de cet algorithme dépendent fortement de la fonction d'évaluation utilisée pour estimer la qualité des positions.

Au cours du développement, plusieurs versions de l'agent ont été testées. Nous avons progressivement amélioré la fonction d'évaluation, ajouté des heuristiques pour mieux gérer les situations d'attaque et de défense, ainsi qu'un mécanisme permettant de détecter certains coups critiques, comme une victoire immédiate ou un blocage nécessaire.

Enfin, nous avons réalisé plusieurs expériences afin de comparer les différentes versions de notre agent et d'analyser l'impact des améliorations apportées.

2 Design de l'agent

2.1 Algorithme de décision

Notre agent repose sur l'algorithme Alpha-Beta, une amélioration du Minimax classique permettant de réduire le nombre de nœuds explorés dans l'arbre de jeu.

L'algorithme est implémenté à travers deux fonctions récursives : `max_value` et `min_value`. La fonction `max_value` cherche à maximiser le score pour notre agent, tandis que `min_value` modélise le comportement de l'adversaire en minimisant ce score. Les actions possibles sont générées avec `Game.actions`

et appliquées avec `Game.apply`. L'exploration s'arrête lorsqu'un état terminal ou une profondeur limite est atteint.

2.2 Gestion du temps

Une gestion du temps est intégrée afin de respecter les contraintes imposées.

La profondeur de recherche est adaptative : elle est fixée à 4 lorsque le temps restant est supérieur à 20 secondes, et réduite à 3 sinon. Cela permet d'explorer plus profondément lorsque le temps le permet, tout en garantissant une réponse rapide dans les situations critiques.

De plus, une condition d'arrêt est utilisée dans les fonctions récursives : si le temps d'exécution dépasse une fraction du temps restant, la recherche est interrompue et une valeur neutre est retournée. Cette stratégie permet d'éviter les dépassements de temps.

2.3 Gestion du premier coup

Lors du premier coup, l'agent sélectionne une action aléatoire parmi les actions possibles.

Ce choix est motivé par le fait que les premières positions du jeu sont souvent symétriques et ne nécessitent pas de calcul approfondi. Cela permet également de réduire le temps de calcul initial.

Cette stratégie est implémentée à l'aide d'un compteur permettant de détecter le premier appel à la fonction `act`.

2.4 Fonction d'évaluation

La fonction d'évaluation est utilisée lorsque la profondeur maximale est atteinte.

Tout d'abord, les états terminaux sont traités en priorité : un score très élevé est attribué en cas de victoire, et un score très faible en cas de défaite. Cela permet de garantir que les résultats finaux sont correctement pris en compte.

Ensuite, l'évaluation repose sur une analyse de fenêtres de taille 4 sur le plateau. Pour chaque fenêtre, le nombre de pions appartenant au joueur et à l'adversaire est compté. Si une fenêtre contient des pions des deux joueurs, elle est ignorée.

Dans le cas contraire, un score est attribué en fonction du nombre de pions alignés. Plus le nombre de pions est élevé, plus le score est important.

Enfin, une heuristique supplémentaire est utilisée pour détecter les coups gagnants au tour suivant. Cette fonction simule les actions possibles et

compte le nombre de positions menant à une victoire immédiate. Cette information est fortement pondérée afin de prioriser les situations critiques.

2.5 Optimisations

Plusieurs optimisations ont été ajoutées pour améliorer les performances de l’agent.

L’élagage Alpha-Beta permet de réduire significativement le nombre de nœuds explorés. Par ailleurs, la détection des coups gagnants immédiats permet d’éviter une exploration inutile dans certaines situations.

Enfin, l’adaptation de la profondeur et la limitation du temps d’exécution contribuent à rendre l’agent plus robuste face aux contraintes du jeu.

3 Évolution de l’agent

3.1 Versions initiales (V1 et V2)

Les premières versions de l’agent reposent sur un algorithme Alpha-Beta avec une fonction d’évaluation simple basée sur des alignements.

Dans V1, l’utilisation d’une pondération exponentielle rendait les décisions instables. Dans V2, la réduction de la profondeur ainsi que certaines transformations des scores ont dégradé les performances en limitant l’anticipation et en perturbant l’ordre des préférences entre les positions.

3.2 Version (V3)

La version 3 a été retenue comme version finale de notre agent. Elle introduit une profondeur adaptative ainsi qu’une fonction d’évaluation structurée basée sur des alignements de taille 4 avec des pondérations explicites.

Cette version offre un bon compromis entre performance et stabilité, avec environ 60% de réussite sur INGINIOUS.

3.3 Versions ultérieures (V4 à V6)

Nous avons exploré plusieurs améliorations intermédiaires visant à renforcer le comportement de l’agent.

Les versions V4 et V5 correspondent à des tests de modification de la fonction d’évaluation (pondérations, pénalités), mais ces approches n’ont pas permis d’améliorer les performances de manière significative.

Dans la version V6, nous avons introduit une heuristique supplémentaire consistant à détecter directement les coups critiques avant l’exécution de

l’algorithme Alpha-Beta. Plus précisément, l’agent vérifie s’il existe un coup menant à une victoire immédiate ou permettant de bloquer une défaite imminente.

Bien que cette approche améliore certaines situations localement, elle perturbe le fonctionnement global de la recherche Alpha-Beta. En effet, ces décisions directes court-circuitent la recherche et introduisent un comportement trop local.

Par conséquent, malgré des améliorations apparentes dans certains cas, cette version s’est révélée moins stable et moins performante globalement.

Cela nous a conduit à conserver la version V3, qui offre un meilleur compromis entre cohérence globale et performance.

4 Expériences

4.1 Configuration expérimentale

Les expériences ont été réalisées en lançant plusieurs parties entre notre agent et différents adversaires à l’aide du script fourni.

Nous avons testé plusieurs configurations en faisant varier l’ordre de jeu (premier ou second joueur), car ce paramètre influence fortement les performances.

Chaque configuration a été exécutée sur plusieurs parties afin d’observer des tendances globales.

4.2 Résultats

Nous avons évalué notre agent en fonction de l’ordre de jeu, car nous avons observé que cela influence fortement les performances.

Configuration	Résultat typique	Observation
Agent joue en premier	Victoire fréquente (60–76%)	Bon comportement offensif
Agent joue en second	Défaite fréquente (10–30%)	Mauvaise gestion défensive

4.3 Comparaison des versions

Afin d’évaluer l’impact des différentes améliorations, nous avons comparé plusieurs versions de notre agent.

Version	Performance typique	Observation
V1–V2	Faible	Décisions instables
V3	~60%	Bon équilibre
V4–V7	10–30%	Déséquilibre attaque/défense

4.4 Analyse des résultats

Ces résultats montrent clairement que les performances de notre agent dépendent fortement de l'ordre de jeu.

En pratique, lorsque notre agent joue en premier, il parvient souvent à imposer une stratégie offensive efficace, ce qui conduit à des taux de victoire relativement élevés (jusqu'à environ 60%).

En revanche, lorsqu'il joue en second, il est beaucoup plus en difficulté. Il ne parvient pas toujours à bloquer les menaces adverses, ce qui entraîne des défaites rapides et des performances faibles (souvent entre 10% et 30%).

Nous avons tenté d'améliorer cet aspect en renforçant la composante défensive de la fonction d'évaluation. Cependant, ces modifications ont souvent dégradé les performances globales de l'agent, en perturbant l'équilibre entre attaque et défense.

Finalement, la version 3 reste la meilleure version obtenue (environ 60%), mais nous n'avons pas réussi à dépasser ce niveau. Les versions suivantes ont systématiquement conduit à une baisse des performances, avec des résultats autour de 30%, 20% ou 10%.

Un résultat important est que notre agent atteint un taux de victoire global de 60% contre l'agent de référence fourni par INGINIOUS.

Cela montre que notre implémentation de l'algorithme Alpha-Beta, combinée à la fonction d'évaluation, permet d'obtenir un agent compétitif.

Cependant, les performances restent variables selon les situations, ce qui indique un manque de robustesse, en particulier dans la gestion des menaces adverses.

5 Discussion

Les résultats obtenus montrent que la performance de notre agent dépend fortement de l'ordre de jeu. Lorsqu'il joue en premier, il est capable d'imposer une stratégie offensive efficace. En revanche, lorsqu'il joue en second, il présente des difficultés à gérer les menaces adverses.

Ces observations mettent en évidence une limite importante de notre fonction d'évaluation, qui reste davantage orientée vers l'attaque que vers la

défense.

Les différentes tentatives d'amélioration ont montré qu'il est difficile de renforcer la composante défensive sans perturber l'équilibre global de l'agent. En effet, certaines modifications ont rendu l'agent trop défensif ou trop dépendant de décisions locales, ce qui a dégradé ses performances.

Le fait d'obtenir 60% de victoire contre l'agent de référence montre que même une approche relativement simple, basée sur Alpha-Beta et une fonction d'évaluation bien structurée, peut être efficace.

Cependant, les tentatives d'amélioration ont montré qu'ajouter des heuristiques supplémentaires ne garantit pas une amélioration, et peut même dégrader les performances si l'équilibre global est perturbé. Cela illustre la difficulté de concevoir une fonction d'évaluation efficace : il ne suffit pas d'ajouter des heuristiques, mais il faut également maintenir un bon équilibre entre les différentes composantes du jeu. Une observation importante est que l'ajout de décisions directes en dehors de l'algorithme Alpha-Beta (comme dans V6) peut améliorer certaines situations locales, mais dégrader la cohérence globale de l'agent.

6 Conclusion

Dans ce projet, nous avons développé un agent pour le jeu Oxono basé sur l'algorithme Alpha-Beta.

Nous avons testé plusieurs versions et introduit différentes améliorations, notamment au niveau de la fonction d'évaluation et de la gestion du temps. Parmi ces versions, la version 3 s'est révélée la plus efficace, offrant un bon compromis entre performance et stabilité.

Cependant, nos résultats montrent que l'agent reste limité, notamment en défense et lorsqu'il joue en second. Les tentatives d'amélioration ont mis en évidence la difficulté de trouver un équilibre entre stratégie offensive et défensive.

Ce projet nous a permis de mieux comprendre les enjeux liés à la conception d'agents adversariaux, ainsi que l'importance des choix de modélisation et d'évaluation.